

2026 전북대학교 프로그래밍 경시대회 JBNUPC

공식 해설

운영진 및 검수진

- | | | |
|-------|------------|-----------------|
| ✓ 이인우 | inwooleeme | 전북대학교 소프트웨어공학과 |
| ✓ 서종호 | | 전북대학교 컴퓨터인공지능학부 |
| ✓ 황지찬 | hwang1535 | 전북대학교 컴퓨터공학부 |
| ✓ 길민준 | soet02 | 전북대학교 컴퓨터공학부 |
| ✓ 송정규 | jung7048 | 전북대학교 컴퓨터공학부 |
| ✓ 홍채운 | hcw889 | 전북대학교 컴퓨터공학부 |
| ✓ 노광희 | naixt1478 | 전북대학교 컴퓨터인공지능학부 |
| ✓ 정재현 | gravekper | 외부 검수 및 출제진 |

번호	문제	의도한 난이도	출제자
A	박테리아 성장 시뮬레이션	Easy	이인우
B	J수 찾기	Easy	서종호
C	거울 평면과 쿼리	Easy	이인우
D	기숙사 배달비 n빵 전략	Medium	황지찬
E	당근 할아버지	Medium	길민준
F	종이 접기와 구멍 뚫기	Medium	송정규
G	태스크 기반 고용 설계	Hard	홍채운
H	Source와 Sink의 집합과 쿼리	Hard	이인우
I	암호화 드론	Challenging	정재헌

A. 박테리아성장시뮬레이션

ad_hoc

출제진 의도 - **Easy**

- ✓ 출제자: 이인우
- ✓ 에디토리얼 작성자: 이인우

A. 박테리아 성장 시뮬레이션

- ✓ 처음에는 크기 $1 = 2^0$ 인 콜로니만 존재한다.
- ✓ 한 번 증식할 때마다 현재 존재하는 모든 콜로니에서 크기가 2배인 새 콜로니가 생긴다.
- ✓ 따라서 i 번 증식이 끝난 뒤에는 크기 2^i 인 콜로니가 새로 등장한다.
- ✓ N 번 증식 후 가능한 크기는 $2^0, 2^1, \dots, 2^N$ 이다.
- ✓ 서로 다른 크기는 총 $N + 1$ 가지이므로, $N + 1$ 을 출력하면 된다.
- ✓ N 은 최대 10^{18} 까지 주어지므로, `int` 대신 `long long` 등의 64비트 자료형을 사용하여야 한다.
- ✓ 시간복잡도는 $\mathcal{O}(1)$ 이다.

B.J수 찾기

brute_force, implementation, math
출제진 의도 - **Easy**

- ✓ 출제자: 서종호
- ✓ 에디토리얼 작성자: 서종호

B. J수 찾기

- ✓ 정의 그대로 각 수가 J수인지 판별하면 된다.
- ✓ n 을 26으로 계속 나누며 나머지를 더하면 $s(n)$ 을 구할 수 있다.
- ✓ 이후 $s(n)$ 이 소수인지, n 이 $s(n)$ 으로 나누어떨어지는지 확인한다.
- ✓ $B \leq 2\,000\,000$ 이고 $s(n)$ 의 값이 작으므로 간단한 소수 판정으로 충분하다.
- ✓ A 부터 B 까지 모든 수에 대해 위 조건을 만족하는 수의 개수를 세면 된다.
- ✓ 시간복잡도는 $\mathcal{O}((B - A + 1) \log_{26} B)$ 이다.

C.거울평면과쿼리

implementation, math

출제진 의도 - **Easy**

- ✓ 출제자: 이인우
- ✓ 에디토리얼 작성자: 이인우

C. 거울 평면과 쿼리

- ✓ 행은 $2N$, 열은 $2M$ 을 주기로 같은 패턴이 반복된다.
- ✓ 좌표를 0-indexed로 바꾸어 $r' = (r - 1) \bmod 2N$ 이라 하자.
- ✓ $r' < N$ 이면 원래 색종이의 r' 행을 보면 된다.
- ✓ $r' \geq N$ 이면 상하가 뒤집힌 영역이므로 $2N - 1 - r'$ 행을 보면 된다.
- ✓ 열도 같은 방식으로 $c' = (c - 1) \bmod 2M$ 을 구한 뒤, $c' \geq M$ 이면 $2M - 1 - c'$ 열로 바꾸면 된다.
- ✓ 각 쿼리마다 계산한 행과 열의 값을 출력하면 된다.
- ✓ 시간복잡도는 전처리 없이 $\mathcal{O}(Q)$ 이다.

D.기숙사배달비 n 빵 전략

sorting, sweeping

출제진 의도 – **Medium**

- ✓ 출제자: 황지찬
- ✓ 에디토리얼 작성자: 황지찬

D. 기숙사 배달비 n 빵 전략

- ✓ 학생들의 방 위치를 오름차순으로 정렬한다.
- ✓ 아직 포함되지 않은 가장 왼쪽 학생의 위치를 X 라 하자.
- ✓ 이 학생을 포함하려면 거점은 $[X - K, X + K]$ 안에 있어야 한다.
- ✓ 이때 거점을 더 왼쪽에 둘 이유는 없으므로, $X + K$ 에 두는 것이 가장 오른쪽까지 덮는 선택이다.
- ✓ 그러면 $[X, X + 2K]$ 안의 학생들을 모두 한 번에 처리할 수 있다.
- ✓ 이후 포함되지 않은 다음 학생부터 같은 과정을 반복한다.
- ✓ 매 단계에서 현재 가장 왼쪽 학생을 포함하면서 가장 오른쪽까지 덮으므로 최적이다.
- ✓ 시간복잡도는 정렬에 의하여 $\mathcal{O}(N \log N)$ 이다.

E. 당근 할아버지

dp, math

출제진 의도 - **Medium**

- ✓ 출제자: 길민준
- ✓ 에디토리얼 작성자: 길민준

E. 당근 할아버지

- ✓ 일반적으로는 i 번째 구역까지 보았고 j 개를 수확했을 때의 최댓값을 DP로 둘 수 있다.
- ✓ 하지만 이를 그대로 계산하면 $\mathcal{O}(NM)$ 이 되어 제한에 맞지 않는다.
- ✓ $V_i \leq 10^6 < 2^{20}$ 이므로 21번째 이후 수확하는 당근의 가치는 항상 0이다.
- ✓ 따라서 점수에 영향을 주는 수확은 앞의 $L = \min(M, 20)$ 번뿐이다.
- ✓ 나머지 $M - L$ 개의 수확은 점수에 영향을 주지 않으므로, 뒤에 충분한 구역만 남아 있으면 된다.

E. 당근 할아버지

- ✓ L 번째 의미 있는 수확 이후에는 $M - L$ 개의 당근을 더 수확해야 한다.
- ✓ 따라서 L 번째 의미 있는 수확 위치는 $P = N - M + L$ 번째 구역 이내여야 한다.
- ✓ 즉, 앞의 P 개 구역에서 정확히 L 개의 당근을 고르는 문제로 바뀐다.
- ✓ $dp[j]$ 를 현재까지 본 당근 중 j 개를 수확했을 때의 최대 가치라 하자.
- ✓ 각 V_i 에 대해 j 를 큰 값부터 갱신한다.

$$dp[j] = \max \left(dp[j], dp[j - 1] + \left\lfloor \frac{V_i}{2^{j-1}} \right\rfloor \right)$$

- ✓ 답은 $dp[L]$ 이며, 시간복잡도는 $\mathcal{O}(20N)$ 이다.

F. 종이 접기와 구멍 뚫기

brute_force, recursion

출제진 의도 - **Medium**

- ✓ 출제자: 송정규
- ✓ 에디토리얼 작성자: 송정규

F. 종이 접기와 구멍 뚫기

- ✓ 각 칸이 접힌 뒤 위에서 몇 번째 층에 오는지 구하면 된다.
- ✓ 좌표를 0-indexed로 두고, 현재 종이의 한 변을 2^n 이라 하자.
- ✓ 절반 길이를 $h = 2^{n-1}$, 한 사분면의 넓이를 $B = h^2$ 라 하자.
- ✓ 한 번 접은 뒤 각 사분면에 더해지는 층 번호의 offset은 다음과 같다.

0	B
$3B$	$2B$

- ✓ 위쪽 행은 $y \geq h$, 왼쪽 열은 $x < h$ 인 사분면을 의미한다.
- ✓ 위쪽 절반은 세로 접기로 y 좌표가 뒤집히고, 오른쪽 절반은 가로 접기로 x 좌표가 뒤집힌다.

F. 종이 접기와 구멍 뚫기

- ✓ 예를 들어 오른쪽 위 사분면이면 offset으로 B 를 더한다.
- ✓ 이후 좌표를 $x = h - 1 - (x - h), y = h - 1 - (y - h)$ 로 바꾼 뒤 크기 2^{n-1} 인 같은 문제를 푼다.
- ✓ 다른 사분면도 표의 offset과 필요한 좌표 반전을 적용하면 된다.
- ✓ 이 과정을 $n = 0$ 이 될 때까지 반복하면 해당 칸의 0-indexed 층 번호를 구할 수 있다.
- ✓ 층 번호가 M 보다 작은 칸이 1층부터 M 층까지 구멍을 뚫었을 때 구멍이 생기는 칸이다.
- ✓ 직사각형의 모든 칸에 대해 이를 판정하면 된다.
- ✓ 시간복잡도는 $\mathcal{O}((x_2 - x_1 + 1)(y_2 - y_1 + 1)N)$ 이다.

G.태스크기반고용설계

bitmask_dp

출제진 의도 - Hard

- ✓ 출제자: 홍채운
- ✓ 에디토리얼 작성자: 홍채운

G. 태스크 기반 고용 설계

- ✓ 전체 배정은 $[1, M]$ 을 여러 연속 구간으로 나누는 문제이다.
- ✓ $maxR[i][s]$ 를 지원자 i 가 s 번 태스크부터 연속으로 맡을 수 있는 가장 오른쪽 태스크 번호라 하자.
- ✓ $maxR[i][s] < s$ 이면 지원자 i 는 s 번 태스크에서 시작할 수 없다.
- ✓ 모든 i, s 에 대해 $maxR[i][s]$ 를 전처리할 수 있다.
- ✓ $costSum[mask]$ 를 $mask$ 에 포함된 지원자들의 비용 합이라 하자.
- ✓ $reach[mask]$ 를 $mask$ 에 포함된 지원자를 모두 한 번씩 사용해 1번부터 연속으로 배정할 수 있는 가장 오른쪽 태스크 번호라 하자.

G. 태스크 기반 고용 설계

- ✓ 초기값은 $reach[0] = 0$ 이다.
- ✓ 도달하지 못한 $mask$ 는 건너뛰고, 현재 $p = reach[mask]$ 라 하자.
- ✓ 아직 고용하지 않은 지원자 i 에 대해 $maxR[i][p + 1] \geq p + 1$ 이면 다음과 같이 갱신한다.

$$reach[mask \mid (1 \ll i)] = \max(reach[mask \mid (1 \ll i)], maxR[i][p + 1])$$

- ✓ 같은 $mask$ 에서는 더 큰 $reach$ 만 남기면 충분하다.
- ✓ 답은 $reach[mask] = M$ 인 $mask$ 중 $costSum[mask]$ 의 최솟값이다.
- ✓ 가능한 $mask$ 가 없다면 -1 을 출력한다.
- ✓ 시간복잡도는 $\mathcal{O}(2^N N + NM^2)$ 이다.
- ✓ 공간복잡도는 $\mathcal{O}(2^N + NM)$ 이다.

H.Source와 Sink의 집합과 쿼리

graph_theory, bipartite_graph, bfs, dfs

출제진 의도 - Hard

- ✓ 출제자: 이인우
- ✓ 에디토리얼 작성자: 이인우

H. Source와 Sink의 집합과 쿼리

- ✓ 문제의 조건에 따르면, 집합 S 에 속한 모든 정점은 Source(모든 간선이 밖으로 나감)이거나 Sink(모든 간선이 안으로 들어옴) 중 하나여야 합니다.
- ✓ S 에 속하지 않은 나머지 정점들에 대해서는 아무런 제약이 없습니다.
- ✓ 이를 파악하기 위해 그래프에 존재하는 임의의 간선 $e = \{u, v\}$ 에 대해, 양 끝점 u, v 가 집합 S 에 포함되는지 여부에 따라 3가지 경우로 나누어 집니다.
 - case 1: $u \notin S, v \notin S$: 두 정점 모두 간선의 방향에 제약을 받지 않으므로 어느 방향이든 상관없습니다.
 - case 2: $u \in S, v \notin S$: u 의 상태(Source/Sink)에 따라 방향을 정하면, 제약이 없는 v 가 이를 수용하므로 문제가 발생하지 않습니다.
 - case 3: $u \in S, v \in S$: 두 정점 모두 제약을 받습니다. 만약 둘 다 Source이거나 둘 다 Sink라면 방향이 충돌합니다.

H. Source와 Sink의 집합과 쿼리

- ✓ 따라서 case 3에서는 반드시 하나는 Source, 다른 하나는 Sink여야만 충돌을 피할 수 있습니다.
- ✓ 집합 S 내부의 정점들끼리 연결된 간선이 있다면 그 간선의 양 끝점은 서로 다른 상태를 가져야 함을 알 수 있습니다.
- ✓ 이는 정점들을 Source 그룹과 Sink 그룹이라는 두 가지 색으로 칠할 때, 인접한 정점끼리는 다른 색을 가져야 한다는 이분 그래프의 정의와 동일합니다.
- ✓ 여기서 이분 그래프가 가지는 아주 중요한 수학적 성질을 짚고 넘어가야 합니다.
- ✓ 정리: 어떤 그래프가 이분 그래프일 필요충분조건은, 그래프 내부에 홀수 길이의 사이클(Odd-length cycle)이 존재하지 않는 것이다.

H. Source와 Sink의 집합과 쿼리

- ✓ 이 성질은 Self-loop가 존재하는 경우 왜 항상 NO가 되는지 설명해 줍니다.
- ✓ Self-loop는 길이가 1인 사이클입니다.
- ✓ 1은 홀수이므로, Self-loop를 포함하는 그래프는 절대 이분 그래프가 될 수 없습니다.
- ✓ 직관적으로 보아도, 어떤 정점 $v \in S$ 에 Self-loop가 있다면 간선의 방향을 어떻게 정하든 v 는 1개의 나가는 간선과 1개의 들어오는 간선을 동시에 가집니다.
- ✓ 즉, 순수한 Source나 Sink가 될 수 없으므로 무조건 모순이 발생합니다.
- ✓ 따라서 주어진 집합 S 에 대해 이분 그래프를 판별해주면 됩니다.
- ✓ 시간복잡도는 쿼리마다 $\mathcal{O}(N + M)$ 입니다.

I. 암호화 드론

segment_tree

출제진 의도 - Challenging

- ✓ 출제자: 정재현
- ✓ 에디토리얼 작성자: 정재현

I. 암호화 드론

- ✓ 드론을 거쳤을 때 메시지가 변환되는 규칙을 0부터 9까지 정수를 나열한 순열 형태의 함수로 정의할 수 있다.
- ✓ 예를 들어, 입력이 그대로 나오는 드론의 경우 $P_0 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ 이고, 0과 1을 서로 교체하는 경우는 $P_1 = \{1, 0, 2, 3, 4, 5, 6, 7, 8, 9\}$ 와 같이 표기할 수 있다.
- ✓ 문제의 2번 쿼리가 요구하는 구간 쿼리는 연속된 드론들의 함수를 합성해 입력값을 적용한 결과를 구하는 것으로 볼 수 있다.

I. 암호화 드론

- ✓ 이러한 함수는 결합법칙이 성립하고 항등원 P_0 이 존재하므로 모노이드이다.
- ✓ 따라서 세그먼트 트리나 제공근 분할법을 사용해 관리할 수 있다.
- ✓ 세그먼트 트리를 사용해 구간 쿼리를 구하면 쿼리 당 $\mathcal{O}(\log n)$, 제공근 분할법을 사용하면 쿼리 당 $\mathcal{O}(\sqrt{n})$ 에 원하는 구간 쿼리 결과를 얻을 수 있다.
- ✓ 1번 쿼리는 두 드론의 위치를 교체한 뒤 자료구조를 갱신하면 된다.